# Agentic SOC

Presentation script and rehearsal notes · Tharun S V

Total speaking time: about 14 minutes including a 2½-minute live demo. Read the bold slide headings as signposts; the paragraphs under them are what you say. Blue boxes are stage directions.

## Before you start

1. Start the API: open a terminal in D:\Work\Max-s-Team and run  python app.py
2. Start the dashboard: in D:\Work\Max-s-Team\frontend run  npm run dev  and open http://localhost:5173
3. Sign in (any username, password of 4+ characters) and press Reset so the console is empty
4. Put your Claude API key in .env if you want the LLM to write real briefings; without it the rule-based fallback is used
5. Keep the explainer video (Agentic_SOC_Explainer.mp4) open in a second window as a backup if the live demo fails
6. Have the project report open at Chapter 8 in case an examiner asks for numbers

## Slide-by-slide script

### Slide 1 · Agentic SOC (title)  (0:30)

Good morning. My project is Agentic SOC — a system that does the job of a Level-1 security analyst automatically: it detects an attack, explains why it is an attack, decides how to contain it, and acts, all in milliseconds, while a human stays in control.

### Slide 2 · The problem  (1:00)

SOC teams face thousands of alerts every day. A Level-1 analyst has to read each alert, decide if it is real, and take the first containment step. That manual step is slow. On top of that, signature-based tools miss new or disguised attacks, and pure machine-learning detectors raise alerts without explaining them, so analysts cannot act on them confidently.

### Slide 3 · Objectives  (0:45)

These were my eight objectives: detection of known attacks, detection of unseen and disguised attacks, threat-intelligence lookup, explanation, automated containment, a live dashboard, analyst override, and learning from analyst feedback.

### Slide 4 · Base paper and research gap  (1:00)

My base paper introduced the CIC-IDS2017 dataset and showed that Random Forest classifies its attacks well. But it stops at detection. Across the twelve papers I surveyed, detection, explanation and response are always treated as separate problems. My project closes that gap with one automated pipeline.

### Slide 5 · Architecture (1:15)

Here is the architecture. Every log goes through the same pipeline in one request: ingest, threat-intelligence lookup, the ML ensemble, SHAP explanation and an adversarial probe check, then LLM reasoning, and finally the response. Only events flagged as anomalous are sent to the LLM, which keeps cost and latency low. Analyst corrections flow back into training.

### Slide 6 · Detection (1:15)

Detection happens in two steps. First the OSINT stage: if an indicator is already known to be malicious, the event is flagged immediately. Otherwise two models score it. The Random Forest catches known attack types. The Isolation Forest is trained only on normal traffic, so it flags anything unusual, which is how we cover zero-day attacks. And because attackers try to blend in, every training attack was copied three times with evasion tricks like port hopping and slow-and-low transfer, so the model learns to see through them.

### Slide 7 · Explainability (SHAP) (1:00)

A key goal was that no decision is a black box. For every alert, SHAP calculates how much each feature pushed the score toward attack or toward normal. On the dashboard, red bars push toward attack and blue toward normal. These top factors are also given to the LLM, so its explanation is grounded in the actual model output.

### Slide 8 · LLM reasoning (1:15)

Flagged events go to the LLM, Claude. The prompt forces a strict JSON answer with fixed fields: the threat level, the attack type, the containment action, an explanation, recommended steps, a YARA rule for future detection, and its reasoning steps. The action must be one of three: isolate the host, revoke credentials, or route the traffic to a honeypot. If the API is down, a rule-based fallback makes the decision, so containment never stops.

### Slide 9 · Containment and learning loop (1:00)

Once the LLM decides, the system writes a firewall playbook for that action — for example a netsh rule that blocks the attacking IP in both directions. The analyst stays in control: any action can be undone, and the analyst can mark an incident as a false positive or confirm it. Those corrections go into a retraining queue, so the model keeps improving.

### Slide 10 · The SOC console — live demo (2:30 (live demo))

This is the React dashboard, connected live to the Flask API.

> *Switch to the live demo here: click Stream demo traffic, then open an incident.*

The overview shows incidents, how many are contained, the median time to contain, and where the time goes in the pipeline.

### Slide 11 · Investigating one incident (1:00)

Opening an incident shows the full investigation: the path through the pipeline with real timings, the SHAP factors, the threat-intelligence match, and the LLM briefing with its

reasoning steps. The attack graph on the right shows which hosts were involved and what action was taken.

### Slide 12 · Results (1:00)

These numbers were measured on the working system. The OSINT lookup takes four hundredths of a millisecond. Scoring plus SHAP takes about 27 milliseconds. A normal event goes through the whole pipeline in about 28 milliseconds, and a flagged event is detected, explained and contained in about 67 milliseconds, not counting the LLM call. Compare that with minutes to hours for a human. I also ran twelve functional test cases against the live API.

### Slide 13 · Status and next steps (0:45)

To summarise where the project stands: the full pipeline works end to end, from ingestion to dashboard. The next steps are a cleaner evaluation on the original CIC-IDS2017 addresses, live threat-intelligence feeds, a persistent database with proper server-side login, a tamper-evident audit log, running playbooks inside a sandbox VM, and optionally a local LLM so that log data never leaves the organisation.

### Slide 14 · Close and questions (0:30)

To conclude: Agentic SOC detects, explains and contains attacks automatically, turning hours of Level-1 triage into milliseconds, while keeping the analyst in control. Thank you. I am happy to take questions.

## Live demo steps (slide 10)

1. Click Stream demo traffic. Say: “I am now sending 33 real log events through the pipeline.”
2. While it runs, point at the incident counter and the Median time to contain tile.
3. When it finishes, click the top incident in Latest incidents.
4. Walk the pipeline trace left to right: threat intel → ML ensemble → SHAP → LLM → containment, reading the millisecond timings.
5. Point at the SHAP chart: “Red pushes toward attack, blue toward normal.”
6. Scroll to the briefing and YARA rule, then click False positive to show the analyst override.
7. Open Learning loop to show the correction waiting for retraining, then Attack graph.
8. Return to the slides.

## Likely questions and honest answers

**Why two models instead of one?**

The Random Forest is supervised, so it only knows attack types it was trained on. The Isolation Forest is trained on normal traffic only, so it flags anything unusual, including attacks that have never been labelled. Together they cover known and unknown attacks.

**How do you stop attackers from evading the model?**

Adversarial training: every training attack was copied three times with evasion tricks — spoofed internal IPs, common ports, slow-and-low transfer, business-hours timing — and kept its attack label. There is also a probe detector that flags one IP sending many near-threshold events.

**What if the LLM is wrong or unavailable?**

The LLM must answer in a fixed JSON format with one of three actions. If the call fails or the JSON is invalid, a rule-based fallback decides. Every action can be undone by the analyst, and corrections feed retraining.

**What accuracy does the model achieve?**

Answer carefully. The current synthetic logs give every attack one source IP and every normal event another, so the model can separate them almost perfectly from the IP alone — the score does not reflect real-world performance. Say: “The pipeline works end to end and the latency is measured; the next step is re-evaluating on CIC-IDS2017 with its original IP addresses so the accuracy reflects real traffic.” Do not quote 96% unless the evaluation has been redone.

**Is it fast enough for real use?**

Measured over 300 events: about 28 ms for a normal event and 67 ms for a flagged one, excluding the LLM call. The LLM runs only on flagged events, which are a small share of traffic.

**What is not built yet?**

Live OSINT feeds, a persistent database with server-side login, a tamper-evident audit log and running playbooks inside a sandbox VM. These are listed as next steps on slide 13.

**Why CIC-IDS2017?**

It is a widely used, realistic intrusion dataset with labelled modern attacks and 80+ flow features, recommended by the Ring et al. survey of 34 IDS datasets.

## 60-second version

If you are short of time: “Agentic SOC is an autonomous Level-1 security analyst. Every log is checked against threat intelligence, scored by a Random Forest and an Isolation Forest hardened with adversarial examples, explained with SHAP, and sent to an LLM that returns a structured containment decision — isolate, revoke or honeypot. The system writes the firewall playbook and shows everything on a live dashboard where analysts can undo actions and correct the model. On the working system a flagged event is detected, explained and contained in about 67 milliseconds, compared with minutes to hours for manual triage.”